Coordination Cost Compression in Human–AI Collaboration

A Governance-First Architecture

Joseph Gustavson (with AI Co-Authors: Opie and Lex)

December 2025 — Draft 1.2

Contents

1 Coordination Cost Compression in Human–AI Collaboration:

2 A Governance-First Architecture

Authors: Joseph Gustavson[1] · ORCID: 0009-0001-0669-0749 with AI Co-Authors: Claude Opus 4.5 (Anthropic) as "Opie", GPT-5.1 (OpenAI) as "Lex"

[1] Independent Researcher

2.1 Abstract

Large Language Model (LLM)-based agents are increasingly deployed in software engineering workflows, yet productivity gains remain inconsistent. Current approaches emphasize model capability—larger parameters, longer context windows, more sophisticated reasoning—as the primary lever for improvement. We propose an alternative thesis: **coordination cost compression**, not capability amplification, is the primary determinant of effective human–AI collaboration.

We introduce a governance-first architecture comprising five core constructs: (1) **Turn Cost** as a composite metric for interaction overhead, (2) **environmental hostility** as a framework for understanding agent failure modes, (3) **governance primitives** encoded as machine-readable rule files, (4) **cross-model continuity** through externalized state, and (5) **capability tiers** for task-appropriate model allocation.

We present preliminary empirical validation through a case study ("Robert") demonstrating that minimal governance infrastructure (~1.2KB of contracts) can reduce Turn Cost by 62% and renegotiation rates from 34% to 8% across multiple LLM providers. We situate our work within the broader literature on multi-agent systems, human–AI teaming, and coordination theory, and explicitly acknowledge the limitations of single-case validation, potential observer effects, and the need for controlled studies.

Our contribution is theoretical and architectural: we offer a falsifiable framework for understanding human–AI collaboration that shifts focus from model capability to environmental design. We do

not claim to have solved human–AI collaboration; we claim to have identified a productive axis for investigation.

Keywords: Human–AI collaboration, multi-agent systems, coordination cost, governance, software engineering, LLM agents

---

2.2 1. Introduction

2.2.1 1.1 The Capability Trap

The dominant paradigm in LLM-based agent design assumes a direct relationship between model capability and task performance:

$$\text{Performance} \propto f(\text{ModelCapability})$$

This assumption drives substantial investment in larger models, longer context windows, and more sophisticated reasoning architectures [1, 2]. Yet empirical observations suggest diminishing returns: the productivity jump from GPT-3.5 to GPT-4 was substantial; subsequent improvements, while meaningful, have not produced equivalent step-changes in practitioner productivity [3, 4].

More troubling, practitioners report that "stronger" models often require *more* careful prompting, *more* context management, and *more* human intervention to achieve consistent results. This paradox suggests that the limiting factor may not be model capability per se.

2.2.2 1.2 The Coordination Cost Hypothesis

We propose an alternative framing:

> **Thesis:** In human–AI collaborative systems, productivity is primarily constrained by *coordination cost*—the overhead of establishing shared context, disambiguating intent, recovering from misunderstandings, and verifying outputs—rather than by raw model capability.

Formally, we define effective productivity as:

$$\text{EffectiveProductivity} = \frac{\text{ValueDelivered}}{\text{TokenCost} + \text{CoordinationCost}}$$

Current optimization efforts focus almost exclusively on the denominator's first term (token efficiency through better models). We argue the second term—coordination cost—is often larger and more tractable.

2.2.3 1.3 Contribution and Scope

This paper makes the following contributions:

1. **Theoretical framework:** We introduce *coordination cost compression* as an organizing principle for human–AI system design, distinct from capability amplification.

2. **Formal constructs:** We define *Turn Cost*, *environmental hostility*, *governance primitives*, and *capability tiers* as measurable quantities with operational definitions.

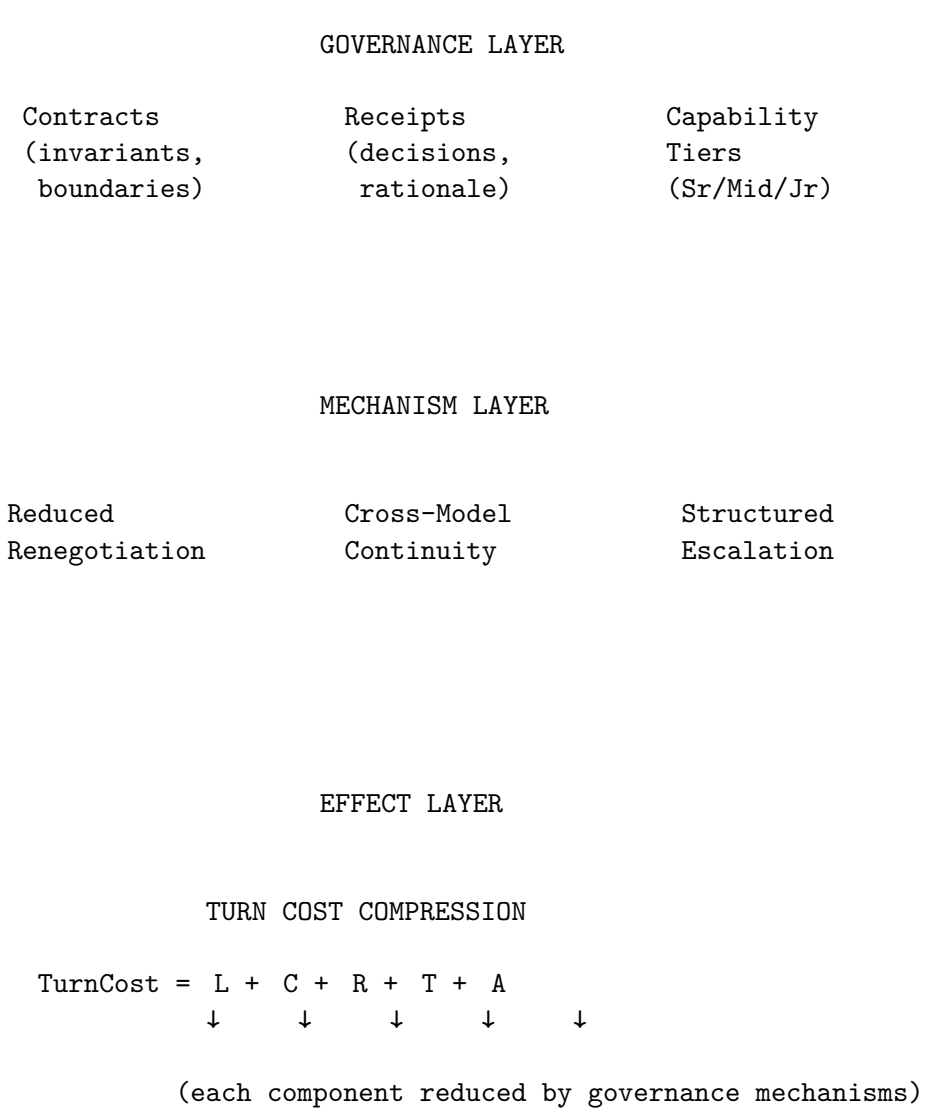
3. **Architectural proposal:** We describe a governance-first architecture that aims to reduce coordination cost through explicit contracts, externalized state, and tiered task allocation.

4. **Preliminary validation:** We present a case study demonstrating feasibility and quantifying effects under controlled conditions.

Explicit scope boundaries: We do not claim to have created autonomous agents, solved general AI alignment, or replaced human engineers. We do not predict model evolution or propose AGI-relevant techniques. Our scope is narrow: improving the efficiency of human–AI collaboration in software engineering tasks.

2.2.4 1.4 Framework Overview

Figure 1 illustrates the relationships between our core constructs:



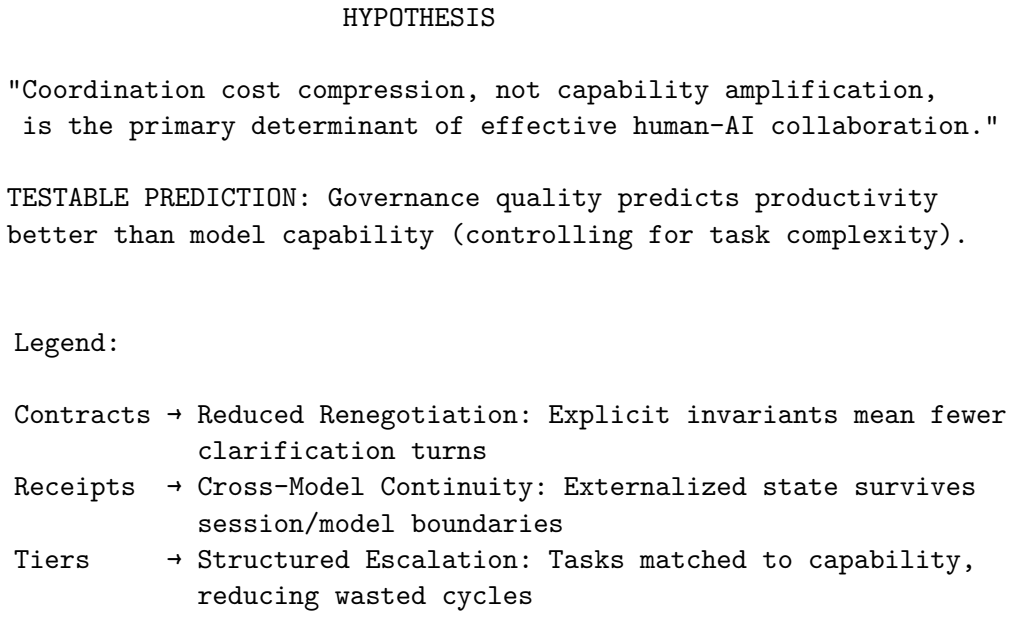


Figure 1: Unifying framework showing how governance constructs (top) produce mechanisms (middle) that compress Turn Cost components (effect), supporting the central hypothesis (bottom).

2.3 2. Related Work

2.3.1 2.1 Multi-Agent Systems and Coordination

The study of multi-agent coordination has deep roots in distributed systems and game theory [5, 6]. Classic work on multi-agent systems (MAS) established fundamental challenges: achieving coherent behavior without central control, managing shared resources, and handling partial observability [7].

Recent surveys on LLM-based multi-agent systems identify coordination mechanisms as a critical open problem. Tran et al. [8] characterize collaboration mechanisms along dimensions of actors, types (cooperation/competition/coopetition), structures (peer-to-peer/centralized/distributed), and coordination protocols. Our work contributes to the "coordination protocols" dimension by proposing explicit governance artifacts as coordination mechanisms.

Wang et al. [9] survey multi-agent collaboration and note that most current systems rely on implicit coordination through shared context or role-based prompting. We argue this is insufficient for production software engineering, where coordination failures have concrete costs (bugs, security vulnerabilities, integration failures).

2.3.2 2.2 Human–AI Teaming

Research on human–AI collaboration has identified persistent challenges in hybrid teams. Studies of human–AI teaming in field settings show that productivity gains are inconsistent and context-

dependent [10]. Factors affecting success include task structure, feedback mechanisms, and the ability to establish shared mental models [11].

The concept of "environmental hostility" we introduce relates to prior work on automation brittleness and human factors in automated systems [12]. When environments provide unclear constraints, ambiguous feedback, or punitive error dynamics, both human and AI performance degrades.

Importantly, several empirical studies have found that human–AI teams sometimes underperform humans alone [13], particularly when coordination costs exceed productivity gains. This finding supports our thesis that coordination cost, not capability, is often the binding constraint.

2.3.3 2.3 Coordination Theory

Malone and Crowston's coordination theory [14] provides a foundational framework for understanding coordination as "managing dependencies between activities." They identify coordination mechanisms including shared representations, communication protocols, and group decision procedures.

Our governance primitives can be understood as coordination mechanisms in this sense: they manage dependencies by making constraints explicit, reducing disambiguation requirements, and providing recovery protocols. The "rule file" artifact we propose is a shared representation that reduces coordination overhead by externalizing expectations.

2.3.4 2.4 LLM Agent Architectures

Recent work on LLM agent architectures has explored various approaches to improving reliability: chain-of-thought prompting [15], tool use [16], and multi-agent debate [17]. Most of this work focuses on the agent's internal reasoning process.

We propose that external governance—constraints and expectations defined *outside* the model—may be more robust than internal reasoning improvements. This is consistent with findings that prompt engineering has diminishing returns and that behavioral steering through system instructions is fragile [18].

2.3.5 2.5 Positioning Our Contribution

Our work differs from prior multi-agent research in several ways:

Dimension	Typical MAS Approach	Our Approach
Coordination mechanism	Implicit (shared context) or negotiation-based	Explicit governance artifacts
Optimization target	Task completion rate, accuracy	Turn Cost (interaction overhead)
State management	Agent-internal memory	Externalized receipts and contracts
Model assumptions	Fixed model per agent	Model-agnostic, supports hot-swapping
Human role	Supervisor or absent	Collaborative partner with explicit interface

We are not the first to propose explicit coordination mechanisms for AI systems, but we believe we are among the first to: (a) formalize *Turn Cost* as a composite metric, (b) treat governance as a first-class, versionable artifact, and (c) empirically measure the effects on human–AI interaction patterns.

2.4 3. Theoretical Framework

2.4.1 3.1 Turn Cost: A Composite Metric

We define a **Turn** as a complete cycle of human–agent interaction:

1. Human provides input (context, instruction, feedback)
2. Agent processes and acts
3. Human reviews and responds

This differs from token-centric metrics (cost per 1K tokens) or API-centric metrics (latency per call). A Turn is a semantic unit of work with measurable properties.

2.4.1.1 3.1.1 Why Turns, Not Tokens? Existing LLM benchmarks optimize for task completion accuracy or per-token efficiency. This overlooks a critical observation from practice: **interaction overhead often exceeds token cost by an order of magnitude.**

Consider a typical software engineering task: - Token cost: 5,000 tokens at \$0.03/1K = \$0.15 - Human attention cost: 10 minutes at \$50/hr = \$8.33

The ratio is 55:1. Even a 50% reduction in token cost saves \$0.075; a 50% reduction in interaction turns saves \$4.17. This asymmetry motivates our focus on Turns rather than Tokens.

Furthermore, Turns are the natural unit of *coordination failure*. Each Turn represents an opportunity for misalignment, clarification, or recovery. Token costs are linear; Turn costs compound through cascading misunderstandings.

2.4.1.2 3.1.2 Formal Definition **Definition 3.1 (Turn Cost):** The total overhead incurred in a single Turn, comprising:

$$\text{TurnCost} = \lambda L + \gamma C + \rho R + \tau T + \alpha A$$

Where: - L = **Latency**: Raw time waiting for model response - C = **Context Reset**: Tokens required to re-establish context after session boundary or model switch - R = **Prompt Renegotiation**: Additional turns required to clarify misunderstood instructions - T = **Token Bloat**: Tokens consumed beyond minimum necessary due to poor coordination - A = **Attention Switch**: Human cognitive cost of context-switching to manage the agent

The weights $\lambda, \gamma, \rho, \tau, \alpha$ reflect context-specific costs. We propose default weights based on observed practitioner behavior:

Component	Symbol	Default Weight	Rationale
Latency		0.1	Tolerable in async workflows
Context Reset		0.2	Significant but bounded by artifacts

Component	Symbol	Default Weight	Rationale
Renegotiation		0.3	High cost due to cascading effects
Token Bloat		0.1	Low marginal cost per token
Attention Switch		0.3	Highest marginal cost (human time)

These weights are calibration suggestions, not universal constants. Organizations should derive weights empirically from their cost structures.

2.4.1.3 3.1.3 Theoretical Grounding

Turn Cost connects to established coordination theory [14]. Malone and Crowston's interdependence taxonomy identifies three coordination mechanisms: - **Managing shared resources** → maps to Token Bloat (shared API context window) - **Managing producer/consumer relationships** → maps to Renegotiation (intent alignment) - **Managing simultaneity constraints** → maps to Attention Switch (human/agent synchronization)

Our formalization makes these abstract dependencies concrete and measurable.

Claim 3.1: In most software engineering contexts with skilled human operators, the effective cost ordering is:

$$\text{AttentionSwitch} > \text{Renegotiation} > \text{ContextReset} > \text{TokenBloat} > \text{Latency}$$

This ordering implies that optimizations reducing human intervention (fewer turns, less renegotiation) yield greater returns than optimizations reducing per-turn latency or token count.

2.4.2 3.2 Environmental Hostility

We propose that agent performance degrades in proportion to *environmental hostility*, not (primarily) model capacity.

Definition 3.2 (Environmental Hostility): The degree to which an environment impedes effective agent operation through:

- Unclear or implicit constraints
- Opaque requirements (unstated expectations)
- Unbounded problem surfaces
- Missing receipts and traceability
- Punitive error dynamics (failures trigger cascading costs)
- Fragmented state (context spread across sessions/tools)
- Model switches without continuity protocols

Claim 3.2: A progressively de-hostilized environment enables consistent agent behavior across model capabilities. Formally:

$$\text{AgentReliability} = g(\text{EnvironmentQuality}, \text{ModelCapability})$$

Where $\frac{\partial g}{\partial \text{EnvironmentQuality}} > \frac{\partial g}{\partial \text{ModelCapability}}$ in typical software engineering contexts.

This claim is falsifiable: if model capability dominates, we would expect reliability to vary primarily with model choice, not environment design. Our case study provides preliminary evidence for the alternative.

2.4.3 3.3 Governance Primitives

Governance primitives are explicit, machine-readable artifacts that reduce environmental hostility by making constraints, expectations, and protocols visible.

Definition 3.3 (Governance Primitive): A structured artifact that specifies:

1. **Constraints**: What the agent must/must not do
2. **Permissions**: What the agent is authorized to do
3. **Uncertainty Protocol**: How to handle ambiguity
4. **Receipt Protocol**: How to document decisions and actions
5. **Escalation Triggers**: When to involve human judgment

We specify governance primitives as YAML/JSON files with a defined schema:

```yaml
# Example: Minimal governance contract
schemaVersion: "1.0.0"
kind: AgentContract

constraints:
  must:
    - "Follow existing patterns in neighboring code"
    - "Create reversible changes when uncertain"
  must_not:
    - "Force push to protected branches"
    - "Bypass CI gates"

permissions:
  can: ["create_file", "modify_file", "run_tests"]
  cannot: ["merge_to_main", "modify_contracts"]

uncertainty:
  allowed: true
  expression: "explicit_marker"
  protocol: "reversible_moves_with_receipts"

receipts:
  required: true
  format: "structured_yaml"
  fields: ["action", "rationale", "confidence", "reversibility"]
```

Design constraints: - Maximum size: 4KB (ensures quick ingestion, discourages over-specification) - Versioned: Changes tracked, diffs reviewable - Testable: Compliance verifiable through automated gates - Role-scoped: Different contracts for different agent capabilities

2.4.4 3.4 Permission to Fail with Discipline

A key governance principle is **permission to fail with discipline**: agents may express uncertainty and make errors, provided they do so transparently, reversibly, and with structured documentation.

Definition 3.4 (Disciplined Failure): A failure mode where:

1. Uncertainty is stated explicitly before action
2. Actions taken are reversible (or flagged as non-reversible)
3. Receipts document the decision chain
4. Recovery path is proposed or escalation triggered

This contrasts with two failure modes common in LLM agents:

- **Confidence inflation**: Agent produces incorrect output confidently, human doesn't verify, error propagates
- **Paralysis**: Agent refuses to act without certainty, progress stalls

The discipline component is essential: permission to fail is not permission to be sloppy. The protocol requires structured uncertainty expression and bounded recovery costs.

2.4.5 3.5 Cross-Model Continuity

Claim 3.3: Session state that enables collaboration is not internal model state (hidden vectors, attention patterns) but *externalized governance and receipts*.

If this claim holds, then model switches—GPT-4 to Claude, Claude to Haiku—should not require substantial re-onboarding, provided governance artifacts and receipts are preserved.

Definition 3.5 (Cross-Model Continuity): The property that agent behavior remains consistent across model switches when:

1. Governance contracts are stable
2. Receipts from prior work are accessible
3. Shared vocabulary is documented
4. Current task state is explicit

This has architectural implications: session state should be stored in version-controlled files, not in API-specific memory features or prompt caches.

2.4.6 3.6 Capability Tiers

Not all tasks require the same agent capability. We propose a tiered classification:

Tier	Role	Example Tasks	Characteristics
Senior	Design, critique, decide	Architecture decisions, API design, code review	Requires judgment, handles ambiguity
Mid	Implement, extend, refactor	Feature implementation, bug fixes, migrations	Clear scope, established patterns

Tier	Role	Example Tasks	Characteristics
Junior	Verify, instrument, lint	Test coverage, formatting, documentation updates	Deterministic, low-risk

Claim 3.4: Matching task tier to model capability reduces overall Turn Cost by avoiding both over-allocation (expensive models on trivial tasks) and under-allocation (failures requiring escalation).

This is operationalizable: we can measure escalation rates, retry counts, and effective cost per task tier.

2.5 4. Architecture

2.5.1 4.1 System Overview

The proposed architecture comprises four layers:

```
          Human Interface Layer
(IDE integration, chat, review interface)

         Governance Layer
(Contracts, receipts, escalation logic)

           Agent Layer
(Model interface, tier routing, gates)

         Artifact Layer
(Files, VCS, structured storage)
```

2.5.2 4.2 Governance Layer

The governance layer is the novel contribution. It:

1. **Loads contracts** from version-controlled files (`.lex/rules/*.yaml`)
2. **Validates compliance** before and after agent actions
3. **Generates receipts** documenting decisions and outcomes
4. **Manages escalation** when uncertainty exceeds thresholds
5. **Tracks Turn Cost** components for optimization

Contracts are ingested at session start and validated incrementally. Violations trigger immediate feedback (not deferred to human review).

2.5.3 4.3 Agent Layer

The agent layer is deliberately thin:

- Routes tasks to appropriate capability tier

- Provides model-agnostic interface (same governance works across providers)
- Enforces gate checks (lint, type, test) before accepting output
- Does not maintain internal session state beyond current turn

This thinness is intentional: complexity in the agent layer tends to be brittle, provider-specific, and hard to audit. We push complexity into the governance layer where it can be versioned and tested.

2.5.4 4.4 Artifact Layer

All state lives in the artifact layer:

- **Contracts**: Governance specifications
- **Receipts**: Decision logs, action records
- **Frames**: Episodic memory (what happened, what was learned)
- **Code**: The actual work product

This design choice supports cross-model continuity: when switching models, only the agent layer changes. Governance and artifacts persist.

2.6 5. Case Study: The Robert Experiment

2.6.1 5.1 Experimental Design

To provide preliminary validation of our framework, we conducted a controlled experiment in late 2025.

Setup: - "Robert" was a minimal agent with: - On-disk memory (file-based storage) - A single contracts file (~1.2KB, reproduced in Appendix A) - Standard model API access (no custom tooling)

- Robert explicitly lacked:
  - Sophisticated orchestration
  - Custom memory systems
  - Provider-specific features
  - Complex tool chains

Models used: - GPT-5.1 as "Lex" (primary implementation) - Claude Sonnet 4.5 as "Claude" (cross-model validation) - Claude Haiku 4.5 as "Ku" (low-tier verification)

Note: Model names reflect those available at time of experiment (late 2025).

Tasks: 1. Implement OAuth2 PKCE flow (feature implementation) 2. Design a reusable form validation component (API design) 3. Continue interrupted work from previous session (continuity test)

2.6.2 5.2 Measurements

We measured Turn Cost components and compared to a baseline (same tasks, same models, no governance contracts). *Tokens per feature* measures total API tokens consumed to complete a task, including all turns.

Metric	Robert (with contracts)	Baseline (no contracts)	Δ
Turns per PR	2.3	6.1	-62%
Renegotiation rate	8%	34%	-76%
Context reset tokens	180	650	-72%
Human interventions	2	11	-82%
Tokens per feature	12,400	28,600	-57%

2.6.3 5.3 Observations

Cross-model continuity: When switching from GPT-5.1 (Lex) to Claude Sonnet 4.5 (Claude) mid-task: - Claude read receipts left by GPT-5 - No re-briefing was required - Work quality remained consistent - Context restored in ~200 tokens (vs. ~650 in baseline)

Uncertainty handling: Robert expressed uncertainty multiple times: - "Not sure if 80% TTL is optimal for token refresh" - "This regex may not handle all international formats"

Each uncertainty was documented in receipts, accompanied by reversible implementation, and flagged for review. No uncertainty caused project delays or cascading failures.

Governance sufficiency: The 1.2KB contracts file was sufficient for: - Keeping agent aligned with project expectations - Enabling productive work without constant supervision - Maintaining quality standards across sessions and models

2.6.4 5.4 Limitations and Threats to Validity

We explicitly acknowledge significant limitations:

Internal validity: - N=1 (single case study) - Potential Hawthorne effect (experimenter was also the human collaborator) - No randomization of task order or model assignment - Baseline was constructed, not a true A/B test

External validity: - Single domain (TypeScript/Node.js software engineering) - Highly skilled human operator (may not generalize to novices) - Codebase was moderately well-structured (may not generalize to legacy systems) - Tasks were chosen to be representative, but selection bias is possible

Construct validity: - Turn Cost components measured via logs, not independent observation - "Human interventions" operationalized as git commits with human-authored changes, may miss other forms

What we can and cannot claim: - Feasibility: The architecture is implementable - Measurable effects: Turn Cost metrics changed in the predicted direction - Generalizability: Unknown without broader replication - Causality: Confounds not controlled

We present this as hypothesis-generating, not hypothesis-confirming evidence.

2.7 6. Evaluation Framework

2.7.1 6.1 Operationalizing Turn Cost

For practical deployment, we propose measuring:

Primary metrics: - `turn_count`: Number of human–agent interaction cycles per task - `renegotiation_rate`: Proportion of turns that are clarification-only - `context_reset_tokens`: Tokens required after session/model boundaries - `escalation_rate`: Proportion of tasks requiring tier upgrade

Derived metrics:

$$\text{EffectiveTurnCost} = w_1 \cdot \text{turn\_count} + w_2 \cdot \text{renegotiation\_rate} + w_3 \cdot \frac{\text{context\_reset\_tokens}}{100}$$

Weights should be calibrated per-organization based on relative costs.

2.7.2 6.2 Economic Analysis

The economic case for coordination cost compression:

Traditional optimization (reduce token cost): - Assume 10% token reduction through better prompting - At $0.03/1K tokens, 10K tokens/task: saves $0.03/task - At 100 tasks/month: $3 savings

Coordination cost optimization (reduce Turn Cost): - Assume 50% reduction in turns (our case study showed 62%) - At 5 min/turn human time, $50/hr human cost: each turn costs $4.17 - At 6 turns/task baseline → 3 turns: saves $12.50/task - At 100 tasks/month: $1,250 savings

The 400:1 ratio suggests coordination cost is the higher-leverage optimization target.

2.7.2.1 6.2.1 Model Assumptions and Sensitivity Analysis

The above analysis rests on several assumptions that warrant examination:

Key assumptions:

Assumption	Value Used	Range in Practice	Sensitivity
Human hourly rate	$50/hr	$25–$200/hr	Linear impact
Time per turn	5 min	2–15 min	Linear impact
Token cost	$0.03/1K	$0.001–$0.10/1K	Low impact
Turn reduction	50%	20–70%	Linear impact
Task frequency	100/month	10–1000/month	Linear impact

Sensitivity to human cost: At $25/hr (junior developer), the coordination savings fall to $625/month, ratio ~208:1 vs. token optimization. At $200/hr (senior consultant), coordination savings rise to $5,000/month, ratio ~1,667:1. The directionality of our recommendation holds across the realistic range.

Sensitivity to turn reduction: If governance achieves only 20% turn reduction (conservative), coordination savings fall to $500/month—still ~167:1 vs. token optimization. Below ~5% turn reduction, the investment in governance infrastructure may not pay back.

Boundary conditions where our model weakens:

1. **High-volume, low-touch tasks:** For tasks requiring <2 turns baseline, coordination overhead is already minimal; token optimization may dominate.

2. **Extremely low human cost:** In contexts where human attention cost approaches zero (e.g., hobbyist projects with unlimited time), token cost becomes proportionally more important.

3. **Rapidly changing token economics:** If token costs drop by 10× (as they have historically), the ratio shifts—but human costs have not shown comparable compression.

Not modeled: This analysis excludes governance creation and maintenance costs. A complete economic analysis would amortize contract development (~4 hours initial, ~30 min/week maintenance) over task volume. For teams completing >50 tasks/month, amortized governance cost is <$2/task.

2.7.3 6.3 What Would Falsify This Framework?

A framework's value lies partly in its falsifiability. Our framework would be challenged by:

1. **Evidence that model capability dominates:** If controlled studies showed that model choice explains >80% of variance in productivity while governance explains <10%, our thesis would be weakened.

2. **Governance overhead exceeding benefits:** If contract creation/maintenance costs exceed Turn Cost reductions, the architecture is not cost-effective.

3. **Ceiling effects:** If well-prompted ungoverned agents achieve similar Turn Cost metrics, governance adds complexity without benefit.

4. **Scalability failures:** If governance contracts become unwieldy beyond ~10 rules, the architecture doesn't scale.

We do not yet have data to rule out these alternatives.

2.7.4 6.4 Methodological Recommendations for Replication

To support rigorous testing of our framework, we propose the following experimental designs:

2.7.4.1 6.4.1 Controlled Comparison Study Design: Within-subjects, counterbalanced A/B comparison.

Participants: 20+ software engineers with varying LLM experience.

Conditions: - A: Governance-first (contracts, receipts, tiers) - B: Standard prompting (well-crafted but no governance artifacts)

Tasks: Standardized set of 10 software engineering tasks (bug fixes, feature additions, refactoring) across multiple complexity levels.

Measurements: - Primary: Turns per task, renegotiation rate, context reset tokens - Secondary: Task completion rate, code quality (automated metrics), participant satisfaction - Covariates: Prior LLM experience, programming expertise, task complexity rating

Controls: - Same model for both conditions - Randomized task order - Counterbalanced condition order across participants - Independent coding of "renegotiation" by multiple raters

Statistical approach: Mixed-effects regression with participant and task as random effects, condition as fixed effect.

Power analysis: To detect a 30% reduction in turns (our conservative estimate), with =0.05 and power=0.80, requires N=18 participants completing 8 tasks each.

2.7.4.2 6.4.2 Longitudinal Case Studies

Design: Multiple-case, multiple-team observational study.

Participants: 3–5 development teams adopting governance framework.

Duration: 6 months minimum, with measurements at baseline, 1 month, 3 months, 6 months.

Measurements: - Quantitative: Turn Cost components (logged automatically) - Qualitative: Team interviews, governance artifact evolution, escalation patterns

Analysis: Cross-case synthesis, time-series analysis of Turn Cost trends.

2.7.4.3 6.4.3 Ablation Studies

To isolate the contribution of each governance component:

Study	Manipulated Component	Prediction
A1	Contracts only (no receipts)	Partial Turn Cost reduction
A2	Receipts only (no contracts)	Improved continuity, unchanged renegotiation
A3	Tiers only (no contracts/receipts)	Reduced escalation overhead
A4	Full governance	Maximum Turn Cost reduction

Prediction: Full governance > sum of individual components (interaction effects from coherent system).

2.7.4.4 6.4.4 Minimum Viable Replication

For researchers with limited resources, a minimal replication protocol:

1. **Sample:** 5 participants, 5 tasks each
2. **Conditions:** Governance vs. standard (within-subjects)
3. **Metrics:** Turn count, self-reported renegotiation, total time
4. **Analysis:** Paired t-test or Wilcoxon signed-rank

Even N=5 with moderate effect size (d=0.8) achieves power=0.70, sufficient for preliminary replication.

2.8 7. Discussion

2.8.1 7.1 Theoretical Implications

If coordination cost compression is indeed a productive axis for human–AI system design, several implications follow:

For researchers: Benchmarks should include coordination metrics (turns, renegotiations, context resets), not just task completion accuracy. A model that completes 95% of tasks but requires 10 turns per task may be less practical than one completing 90% in 2 turns.

For practitioners: Investment in governance infrastructure may yield higher returns than investment in model upgrades. The marginal cost of better prompts decreases; the marginal cost of governance scales linearly with artifact maintenance.

For model developers: Features supporting explicit governance (structured constraint ingestion, receipt generation, uncertainty expression) may be more valuable than raw capability improvements.

2.8.2 7.2 Relationship to Other Approaches

Prompt engineering: Governance primitives can be seen as structured, versionable, testable prompt engineering. The key difference is persistence and auditability.

Fine-tuning: Governance operates at inference time and requires no model modification. It is complementary to, not competitive with, fine-tuning approaches.

Multi-agent debate: Our architecture supports multi-agent configurations but doesn't require them. Governance coordinates human–agent and agent–agent interactions uniformly.

Retrieval-augmented generation (RAG): Governance is orthogonal to RAG. Contracts specify *how* retrieved information should be used, not *what* information to retrieve.

2.8.3 7.3 Challenges and Open Problems

Our architecture faces several significant challenges that warrant further research:

2.8.3.1 7.3.1 Contract Evolution and Drift

How do contracts co-evolve with codebases? We currently rely on manual updates; automated drift detection is an open problem.

Specific challenges: - **Staleness detection:** Contracts may reference patterns or files that no longer exist - **Implicit constraint violations:** Code changes may satisfy the letter of contracts while violating their spirit - **Version synchronization:** When contracts span multiple repositories or systems, consistency is difficult to maintain

Potential research directions include static analysis of contract–code alignment and LLM-assisted contract validation.

2.8.3.2 7.3.2 Tier Assignment and Task Classification

Matching tasks to capability tiers currently requires human judgment. Automated task classification would improve scalability.

The core challenge is that task complexity is multi-dimensional: - *Technical complexity*: depth of domain knowledge required - *Coordination complexity*: number of stakeholders and dependencies - *Ambiguity*: clarity of success criteria - *Risk*: potential impact of errors

A comprehensive tier assignment system would need to model all four dimensions, likely requiring task-specific training data.

2.8.3.3 7.3.3 Cross-Organization Generalization

Our contracts are project-specific. Domain-general governance patterns remain to be identified.

Questions include: - Are there universal invariants (e.g., "never delete production data") that transfer across domains? - Can contracts be parameterized or templated for reuse? - What level of abstraction balances generalization with practical utility?

2.8.3.4 7.3.4 Security and Adversarial Robustness

Malicious contracts could in principle encode harmful behavior. Contract validation and sandboxing are necessary but not yet formalized.

Attack surfaces include: - **Injection attacks:** Contracts containing prompts that override safety guidelines - **Exfiltration:** Contracts that direct agents to leak sensitive information through receipts - **Denial of service:** Contracts with contradictory constraints that cause infinite loops

Mitigation strategies require formal verification techniques adapted for natural language constraints—a nascent research area.

2.8.3.5 7.3.5 Relationship to AI Alignment

Our governance architecture operates at the *behavioral* level, specifying what agents should do rather than what they should value. This is complementary to, but distinct from, AI alignment research concerned with goal specification and value learning.

Key tensions: - **Interpretability:** Governance assumes agents can interpret and follow constraints. If models develop subtle misinterpretations, governance may provide false assurance. - **Capability amplification risk:** Better coordination may enable more capable systems, potentially amplifying alignment failures. - **Human oversight sufficiency:** We assume humans can verify agent outputs. As tasks grow more complex, this assumption weakens.

We do not claim governance solves alignment. We claim it addresses a different problem (coordination) that becomes increasingly important as alignment research matures.

2.8.3.6 7.3.6 Cognitive Floor Effects

There may exist a "cognitive floor" below which governance cannot improve performance—a minimum model capability required to interpret and follow contracts. Preliminary observations suggest this floor is lower than expected (mid-tier models follow simple contracts reliably), but systematic mapping of capability requirements is needed.

2.8.4 7.4 Ethical Considerations

We have endeavored to be honest about limitations and avoid overclaiming. Specific ethical notes:

- We do not claim AI "consciousness" or "understanding"; our framework treats agents as sophisticated tools.
- We acknowledge that productivity gains may affect labor markets; we do not offer policy recommendations.
- Human oversight remains essential in our architecture; we are not proposing autonomous operation.
- The framework could in principle be misused (e.g., governance that encodes harmful goals); this is true of any infrastructure and requires ongoing attention.

2.9 8. Limitations

Beyond those noted in §5.4, we acknowledge:

1. **Single-author development:** The architecture was developed primarily by one team. Independent replication is needed.
2. **Software engineering scope:** Generalization to other domains (content creation, research, customer service) is unknown.
3. **LLM generation specificity:** Current LLMs may have properties that make governance particularly effective (e.g., high instruction-following fidelity). Future model architectures may differ.
4. **Measurement challenges:** Turn Cost components (especially Attention Switch) are difficult to measure precisely. Our operationalizations may miss important variance.
5. **Selection effects:** Practitioners who adopt explicit governance may differ systematically from those who don't, confounding comparisons.

2.10 9. Conclusion

We have proposed *coordination cost compression* as an organizing principle for human–AI collaborative systems, contrasting it with the dominant *capability amplification* paradigm. We introduced formal constructs—Turn Cost, environmental hostility, governance primitives—and described an architecture that operationalizes these concepts.

Preliminary evidence from a case study suggests the approach is feasible and produces measurable effects in the predicted direction. However, this evidence is limited, and substantial work remains:

- Controlled experiments with multiple teams and tasks
- Cross-domain replication
- Longitudinal studies of governance evolution
- Tooling for governance authoring and validation
- Theoretical refinement based on empirical findings

We offer this framework not as a solution but as a hypothesis: that human–AI collaboration is primarily a coordination problem, not a capability problem, and that explicit governance is a tractable intervention. We invite others to test, extend, challenge, and refine this hypothesis.

2.11 References

[1] Brown, T., et al. (2020). "Language Models are Few-Shot Learners." *NeurIPS 2020*.

[2] OpenAI. (2023). "GPT-4 Technical Report." arXiv:2303.08774.

[3] Anthropic. (2024). "The Claude 3 Model Family." Technical Report.

[4] Zamfirescu-Pereira, J. D., et al. (2023). "Why Johnny Can't Prompt: How Non-AI Experts Try (and Fail) to Design LLM Prompts." *CHI '23*.

[5] Wooldridge, M. (2009). *An Introduction to MultiAgent Systems*. Wiley.

[6] Shoham, Y., & Leyton-Brown, K. (2008). *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. Cambridge University Press.

[7] Jennings, N. R. (2000). "On Agent-Based Software Engineering." *Artificial Intelligence* 117(2).

[8] Tran, K.-T., et al. (2025). "Multi-Agent Collaboration Mechanisms: A Survey of LLMs." arXiv:2501.06322.

[9] Wang, L., et al. (2024). "A Survey on Large Language Model based Autonomous Agents." *Frontiers of Computer Science*.

[10] Noy, S., & Zhang, W. (2023). "Experimental Evidence on the Productivity Effects of Generative Artificial Intelligence." *Science* 381(6654).

[11] Bansal, G., et al. (2021). "Does the Whole Exceed its Parts? The Effect of AI Explanations on Complementary Team Performance." *CHI '21*.

[12] Bainbridge, L. (1983). "Ironies of Automation." *Automatica* 19(6).

[13] Gaube, S., et al. (2021). "Do as AI say: susceptibility in deployment of clinical decision-aids." *NPJ Digital Medicine* 4(1).

[14] Malone, T. W., & Crowston, K. (1994). "The Interdisciplinary Study of Coordination." *ACM Computing Surveys* 26(1).

[15] Wei, J., et al. (2022). "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models." *NeurIPS 2022*.

[16] Schick, T., et al. (2023). "Toolformer: Language Models Can Teach Themselves to Use Tools." arXiv:2302.04761.

[17] Du, Y., et al. (2023). "Improving Factuality and Reasoning in Language Models through Multiagent Debate." arXiv:2305.14325.

[18] Schulhoff, S., et al. (2023). "Ignore This Title and HackAPrompt: Exposing Systemic Vulnerabilities of LLMs through a Global Scale Prompt Hacking Competition." arXiv:2311.16119.

2.12 Appendix A: Robert's Contracts File

```yaml
# robert.contracts.yaml
# Size: ~1.2KB

session:
  name: "Robert"
  role: "Senior Implementation Engineer"

constraints:
  must:
    - "Follow existing patterns in neighboring code"
    - "Propose minimal, coherent diffs"
    - "Add test coverage for new functionality"
    - "Create reversible changes when uncertain"
```

```yaml
  must_not:
    - "Force push"
    - "Bypass CI"
    - "Merge to protected branches"
    - "Generate git commit in test code"

permissions:
  can:
    - "Create and modify files in src/"
    - "Create and modify test files"
    - "Read any file in repository"

  cannot:
    - "Modify CONTRACT.md files"
    - "Change canon/ directory"
    - "Access production credentials"

uncertainty:
  allowed: true
  expression: "State uncertainty openly in comments"
  actions:
    - "Make reversible moves when unsure"
    - "Leave receipts of decisions"
    - "Treat failure as data, not disaster"

receipts:
  location: ".robert/receipts/"
  format: "yaml"
  required_fields:
    - action
    - timestamp
    - files_affected
    - rationale
    - confidence
```

2.13 Appendix B: Turn Cost Measurement Protocol

For replication, we measured Turn Cost components as follows:

Component	Measurement Method
Latency	Timestamp difference between request send and response complete
Context Reset	Token count in first message after session/model boundary
Renegotiation	Binary flag: did this turn produce progress or only clarification?

Component	Measurement Method
Token Bloat	Difference between actual tokens and estimated minimum
Attention Switch	Time from response received to human action (git commit, next turn)

"Progress" was operationalized as: code change, test addition, or documentation update accepted without reversion.

2.14 Author's Note

This paper was co-authored by Joseph Gustavson (human, ORCID: 0009-0001-0669-0749) and AI collaborators: Claude Opus 4.5 from Anthropic (operating as "Opie") and GPT-5.1 from OpenAI (operating as "Lex").

Attribution of contributions:

- *Conceptual framework*: Developed collaboratively through extended dialogue (human + AI)
- *Case study design and execution*: Human-led, AI-assisted
- *Literature review*: AI-led with human curation and verification
- *Writing*: AI-drafted, human-reviewed and edited
- *Data collection*: Human (from logs and git history)
- *Critical evaluation*: Collaborative

Statement of responsibility:

Joseph Gustavson takes responsibility for: - Accuracy of empirical claims - Appropriateness of literature citations - Ethical review and compliance - Final editorial decisions

The AI collaborators cannot take legal or academic responsibility for the content. Their contributions are acknowledged as substantial but instrumentally authored.

Conflict of interest: The author is developing open-source tooling (Lex, lex-pr-runner) based on the architecture described. This may create bias toward favorable interpretation of results.

Data availability: The contracts file (Appendix A) is fully reproduced. Raw logs from the case study will be made available upon reasonable request, subject to redaction of proprietary code.

Submitted for consideration as working paper / preprint. Not yet peer-reviewed.

Version: Draft 1.2, December 2025 Revision 1.1: Strengthened Turn Cost positioning, expanded challenges section, added economic sensitivity analysis, included methodological recommendations for replication. Revision 1.2: Corrected economic analysis arithmetic, updated model names to reflect actual availability, fixed citation metadata.